```
> # Appendix B.1: coverage and rigorous derivative enclosures.
  # All arithmetic is exact rational arithmetic.
  # The output is used by the lower-bound calculation in B.2--B.3.

  B1_GRID  := 10^30:
  B1_ALPHA := [41/25, 33/20]:
  B1_BETA  := 13/10000:

  # Outward rounding to multiples of 10^(-30).
  B1_Down := proc(x::rational)
      return floor(B1_GRID*x)/B1_GRID;
  end proc:

  B1_Up := proc(x::rational)
      return -floor(-B1_GRID*x)/B1_GRID;
  end proc:

  # An interval is represented by [lower endpoint, upper
  endpoint].
  B1_Add := proc(A::list, B::list)
      return [
          B1_Down(A[1] + B[1]),
          B1_Up(A[2] + B[2])
      ];
  end proc:

  B1_Mul := proc(A::list, B::list)
      local products;

      products := [
          A[1]*B[1], A[1]*B[2],
          A[2]*B[1], A[2]*B[2]
      ];

      return [
          B1_Down(min(op(products))),
          B1_Up(max(op(products)))
      ];
  end proc:

  # Rigorous bounds for exp(x), using the degree-24 Taylor sum
  # and the geometric upper bound for its remainder.
  B1_ExpAt := proc(x::rational)
      local term, total, r, nextTerm, tailUpper;

      if x < 0 or x > 13/5 then
```

```
        error "exponential argument outside [0,13/5]: %1", x;
    end if;

    if x/26 >= 1 then
        error "invalid geometric-tail ratio: %1", x/26;
    end if;

    term  := 1;
    total := 1;

    for r from 1 to 24 do
        term  := term*x/r;
        total := total + term;
    end do;

    nextTerm := term*x/25;
    tailUpper := nextTerm/(1 - x/26);

    return [
        B1_Down(total),
        B1_Up(total + tailUpper)
    ];
end proc:

# Polynomial coefficients in ascending powers of z.
# P[j](z) represents v^(j)(w) when z = tanh(w/2).
# Recurrence: P[j+1] = (1-z^2)*P[j]'/2.
B1_Polynomials := proc()
    local z, p, polynomials, j, k;

    polynomials := Array(0..5, datatype=anything);
    p := (1 - z^2)/4;

    for j from 0 to 5 do
        polynomials[j] := [
            seq(coeff(p, z, k), k = 0..degree(p, z))
        ];

        if j < 5 then
            p := expand((1 - z^2)*diff(p, z)/2);
        end if;
    end do;

    return polynomials;
end proc:
```

```
# Horner evaluation with outward-rounded interval operations.
B1_Horner := proc(coefficients::list, X::list)
    local answer, k;

    answer := [0, 0];

    for k from nops(coefficients) by -1 to 1 do
        answer := B1_Add(
            B1_Mul(answer, X),
            [coefficients[k], coefficients[k]]
        );
    end do;

    return answer;
end proc:

B1_Verify := proc()
    local data, polynomials, cellIndex, W,
          Eleft, Eright, Tint, V, j, previous,
          cellCount, expCount, derivativeCount;

    data := Array(0..2599, datatype=anything);
    polynomials := B1_Polynomials();

    previous := 0;
    cellCount := 0;
    expCount := 0;
    derivativeCount := 0;

    for cellIndex from 0 to 2599 do

        W := [cellIndex/1000, (cellIndex + 1)/1000];

        # Check adjacency and the exact width of every cell.
        if W[1] <> previous
           or W[2] - W[1] <> 1/1000 then
            error "coverage failure at cell %1", cellIndex;
        end if;

        Eleft  := B1_ExpAt(W[1]);
        Eright := B1_ExpAt(W[2]);

        if Eleft[1] < 1 or Eleft[1] > Eleft[2]
           or Eright[1] < 1 or Eright[1] > Eright[2] then
            error "invalid exponential enclosure at cell %1",
                  cellIndex;
```

```
        end if;

        expCount := expCount + 2;

        # exp(w) and (x-1)/(x+1) are increasing.
        # Hence this interval encloses tanh(w/2) on all of W.
        Tint := [
            B1_Down((Eleft[1] - 1)/(Eleft[1] + 1)),
            B1_Up((Eright[2] - 1)/(Eright[2] + 1))
        ];

        if Tint[1] < 0 or Tint[1] > Tint[2]
           or Tint[2] >= 1 then
            error "invalid tanh enclosure at cell %1",
cellIndex;
        end if;

        # V[j+1] encloses v^(j)(w), j = 0,...,5,
        # throughout the entire closed interval W.
        V := [
            seq(B1_Horner(polynomials[j], Tint), j = 0..5)
        ];

        for j from 1 to 6 do
            if not type(V[j][1], rational)
               or not type(V[j][2], rational)
               or V[j][1] > V[j][2] then
                error "invalid derivative enclosure: cell %1,
order %2",
                      cellIndex, j - 1;
            end if;

            derivativeCount := derivativeCount + 1;
        end do;

        # The positive lower endpoint is needed in B.2.
        if V[1][1] <= 0 then
            error "nonpositive lower bound for v at cell %1",
                  cellIndex;
        end if;

        # Store [w interval, tanh interval, derivative
intervals].
        data[cellIndex] := [W, Tint, V];

        previous := W[2];
```

```
            cellCount := cellCount + 1;
        end do;

        if previous <> 13/5 or cellCount <> 2600
           or expCount <> 5200 or derivativeCount <> 15600 then
            error "incorrect domain coverage or verification count";
        end if;

        print(["B1_COVERAGE_PASS", cellCount]);
        print(["B1_EXPONENTIAL_ENCLOSURES_PASS", expCount]);
        print(["B1_DERIVATIVE_ENCLOSURES_PASS", derivativeCount]);
        print("APPENDIX_B1_PASS");

        return data;
    end proc:

    # Run the certificate and retain its interval data.
    B1Data := B1_Verify():
```

```
                    ["B1_COVERAGE_PASS", 2600]
            ["B1_EXPONENTIAL_ENCLOSURES_PASS", 5200]
            ["B1_DERIVATIVE_ENCLOSURES_PASS", 15600]
                    "APPENDIX_B1_PASS"
```

```
>
```